Дополнительные возможности

Домен

- **Домен** — это определяемый пользователем тип данных, который может иметь необязательные ограничения (DEFAULT, NOT NULL и CHECK), ограничивающие допустимый набор значений
- Домены полезны для абстрагирования и вынесения общих характеристик разных полей в единое место для упрощения сопровождения
  - Например, столбец телефон, определенный в нескольких таблицах, требует наличие ограничения CHECK, проверяющие синтаксис телефона
  - В этом случае лучше определить домен, а не задавать для каждой таблицы отдельные ограничения

https://postgrespro.ru/docs/postgresql/14/sql-createdomain

Создание домена

- Для создания домена используется оператор **CREATE DOMAIN**
  - Имя должно быть уникально в пределах схемы среди имён типов и доменов
  - Необходимо иметь право **USAGE** для исходного типа данных
  - Пользователь, определяющий домен, становится его владельцем

```
CREATE DOMAIN схема.имя [ AS ] тип_данных
    [ COLLATE правило_сортировки ]
    [ DEFAULT выражение ]
    [ CONSTRAINT имя_ограничения { NOT NULL | NULL | CHECK (выражение) }]
```

- ***DEFAULT***:
  - Если значение по умолчанию не указано, им будет значение NULL
  - Значение по умолчанию для конкретного столбца имеет приоритет
- ***CHECK***:
  - Задает ограничение проверки **<u>уровня столбца</u>**
  - Проверяемое значение в этом выражении обозначается ключевым словом **VALUE**
  - Если для домена задано несколько ограничений **CHECK**, они будут проверяться в алфавитном порядке имён

Пример

- Фамилия и имя студента не должны содержать NULL и пробелы

```sql
CREATE TABLE marksheet (
    student_id SERIAL PRIMARY KEY,
    first_name VARCHAR NOT NULL,
    last_name  VARCHAR NOT NULL,
    email VARCHAR NOT NULL,
    marks_obtained INT NOT NULL,
    CHECK (
        first_name !~ '\s'
        AND last_name !~ '\s'
    )
);
```

```sql
CREATE DOMAIN student_detail
AS
    VARCHAR NOT NULL CHECK (VALUE !~ '\s');

CREATE TABLE marksheet (
    student_id serial PRIMARY KEY,
    first_name student_detail,
    last_name  student_detail,
    marks_obtained INT NOT NULL,
    email VARCHAR NOT NULL
);
```

Изменение домена

```
ALTER DOMAIN имя { SET DEFAULT выражение | DROP DEFAULT }
ALTER DOMAIN имя { SET | DROP } NOT NULL
ALTER DOMAIN имя ADD ограничение_домена [ NOT VALID ]
ALTER DOMAIN имя DROP CONSTRAINT [ IF EXISTS ] имя_ограничения [ RESTRICT | CASCADE ]
ALTER DOMAIN имя RENAME CONSTRAINT имя_ограничения TO имя_нового_ограничения
ALTER DOMAIN имя VALIDATE CONSTRAINT имя_ограничения
ALTER DOMAIN имя OWNER TO { новый_владелец | CURRENT_USER | SESSION_USER }
ALTER DOMAIN имя RENAME TO новое_имя
ALTER DOMAIN имя SET SCHEMA новая_схема
```

- В случае, если в выражении CHECK используется пользовательская функция, поведение которой впоследствии меняется необходимо:
  - удалить ограничение (ALTER DOMAIN ….DROP CONSTRAINT)
  - изменить определение функции
  - пересоздать ограничение (при этом будет выполнена перепроверка сохранённых данных)

https://postgrespro.ru/docs/postgresql/14/sql-alterdomain

Удаление домена

```
DROP DOMAIN [ IF EXISTS ] имя [, ...] [ CASCADE | RESTRICT ]
```

- **CASCADE**
  - Автоматически удалять объекты, зависящие от данного домена (например, столбцы таблиц), и, в свою очередь, все зависящие от них объекты
- **RESTRICT**
  - Отказать в удалении домена, если от него зависят какие-либо объекты. Это поведение по умолчанию
- **Удалить домен может только его владелец**

Ограничение EXCLUDE

- **EXCLUDE** определяет ограничение исключения, которое гарантирует, что при сравнении любых двух строк в указанных столбцах или выражениях с использованием указанных операторов хотя бы одно из этих сравнений (**OR**) вернет значение **FALSE** или **NULL**
  - **Если хотя бы одно условие не выполняется – запись будет добавлена**
  - Если все указанные операторы проверяют равенство, это эквивалентно ограничению **UNIQUE** (но ограничение UNIQUE работает быстрее)
- Когда добавляется новая или обновленная строка, необходимо сравнить ее с каждой существующей строкой на основе списка сравнений, указанных в ограничении:
  - **Если все сравнения между новыми данными и существующей строкой возвращают значение TRUE, отклонить новые данные**

Синтаксис

```
[CONSTRAINT constraint_name] EXCLUDE [ USING index_method ]
(name_of_column WITH operator [, ... ] )
index_parameters [ WHERE ( predicate ) ]
```

- **Ограничения исключения реализуются с помощью индекса**
  - на практике метод доступа всегда будет **GiST** или **SP-GiST**
- Необходимо установить расширение **btree_gist**, которое определяет ограничения исключения для простых скалярных типов данных (выполнить один раз для каждой БД)

```
CREATE EXTENSION btree_gist;
```

- Предикат позволяет указать ограничение исключения для подмножества записей таблицы (создает частичный индекс)
  - Обратите внимание, что круглые скобки необходимы вокруг предиката

Пример 1

- **Задача**: не позволять нескольким пользователям одновременно арендовать одну и ту же недвижимость (room_id).
  Можно арендовать ту же недвижимость на следующей неделе или другую недвижимость на перекрывающийся период
- Новая строка резервирования будет недействительна, если:
  - значение **room_id**, равно значению **property_id** в существующей строке (**WITH =**)
  - и имеет диапазон дат (**tsrange**), который пересекается (**&&**) с диапазоном дат существующего резервирования (**WITH &&**)
    (граничные даты должны быть включены в диапазон - '[]‘)

```
ALTER TABLE reservations ADD CONSTRAINT no_overlapping_rentals
EXCLUDE USING gist (property_id WITH =,
tsrange("checkin_time", "checkout_time", '[]') WITH &&);
```

Пример 2

```
CREATE EXTENSION btree_gist;

CREATE TABLE employee (
   ID INT PRIMARY KEY     NOT NULL,
   NAME           TEXT,
   AGE            INT  ,
   ADDRESS        CHAR(50),
EXCLUDE USING gist
-- USING gist – это тип индекса,
--который нужно создать и использовать для
--принудительного применения
--исключить тех:
  ( NAME WITH =,
--чье имя совпадает с уже существующим
    AGE WITH <>) );
--а возраст при этом не совпадает
```

Новая запись будет ***недействительна*** если имя сотрудника совпадает с именем уже существующего **И** при этом возраст не совпадает

```
INSERT INTO employee
VALUES (1, 'Paul', 32, 'California' );
INSERT INTO employee
VALUES (2, 'Paul', 32, 'Texas');
INSERT INTO employee
VALUES (3, 'Paul', 42, 'California');
```

SQL Error [23P01]: ERROR: conflicting key value violates exclusion constraint "employee_name_age_excl"
Подробности: Key (name, age)=(Paul, 42) conflicts with existing key (name, age)=(Paul, 32).

Дополнительное чтение

- https://www.tutorialspoint.com/postgresql/postgresql_constraints.htm
- https://stackoverflow.com/questions/10292355/how-to-create-a-real-one-to-one-relationship-in-sql-server
- https://habr.com/ru/articles/526042/